

Software Requirements Specification: Project L.O.V.E. (Listener - Observer - Versor - Experience)

1. Executive Summary

The domain of Affective Computing has long been constrained by two-dimensional models that reduce the complexity of the human experience to static Cartesian coordinates. Project L.O.V.E. (Listener - Observer - Versor - Experience) represents a fundamental paradigm shift in this field. It proposes a system that maps human emotion not as fixed points on a grid, but as dynamic *orientations* within a three-dimensional conceptual space. By synthesizing the rigorous qualitative research of Dr. Brené Brown's *Atlas of the Heart* with the non-commutative algebraic framework of quaternions, the system seeks to quantify the "work" required to shift between emotional states.

The central thesis of Project L.O.V.E. is that emotions are not destinations but vectors of orientation. A transition from "Anger" to "Grief" is not a linear translation but a complex rotation involving changes in Valence (hedonic tone), Arousal (energy), and Connection (relational alignment). The system architecture is designed to capture, process, and visualize these transitions in real-time. It employs a microservices architecture—The L.O.V.E. Stack—comprising four distinct modules: the **Listener** for multi-modal ingestion and normalization; the **Observer** for contextual state management and semantic vector storage; the **Versor** for pure mathematical processing using quaternion logic; and the **Experience** for high-fidelity, 3D biometric visualization.

This document serves as the exhaustive Software Requirements Specification (SRS) for Project L.O.V.E. It details the theoretical underpinnings, mathematical models, architectural decisions, and implementation strategies required to build a system capable of mapping the 87 distinct emotions identified in *Atlas of the Heart*.¹ It addresses the technical challenges of real-time audio transcription using edge-optimized models³, the complexities of high-dimensional vector indexing in PostgreSQL⁴, and the performance optimization required for rendering 3D graphics on mobile devices using React Three Fiber.⁵ The result is a blueprint for a system that does not merely track mood, but visualizes the trajectory of the human soul.

2. Theoretical Framework: The Digital Atlas of the Heart

2.1 The Limitations of Existing Models

Traditional affective computing relies heavily on the Valence-Arousal-Dominance (VAD) model. While effective for basic sentiment analysis, this model fails to capture the nuance of complex human experiences. Specifically, the dimension of "Dominance"—often defined as the degree of control one feels over a situation—is insufficient for mapping the relational depth of emotions described by Brené Brown. In Brown's framework, the primary driver of the human experience is not control, but connection.¹

Project L.O.V.E. replaces the "Dominance" axis with "Connection" (\$z\$-axis). This creates a new dimensional space: the **VAC Model** (Valence, Arousal, Connection).

- **Valence (\$x\$):** The intrinsic attractiveness (positive) or aversiveness (negative) of an emotion.
- **Arousal (\$y\$):** The physiological state of being awoken or stimulated to action.
- **Connection (\$z\$):** The degree of authentic alignment with oneself or others.

This shift allows the system to distinguish between emotions that might have similar Valence and Arousal but opposite relational vectors. For example, "Pity" and "Compassion" might both be responses to suffering, but Pity is defined by separation (negative Connection), while Compassion is defined by shared humanity (positive Connection).⁶

2.2 The 13 Categories of Emotion

The Observer module is founded on the "Atlas," a comprehensive database of 87 emotions organized into 13 categories. These categories, or "Places We Go," serve as the semantic clustering mechanism for the system.¹

2.2.1 Places We Go When Things Are Uncertain or Too Much

This category includes Stress, Overwhelm, Anxiety, Worry, Avoidance, Excitement, Dread, Fear, and Vulnerability.

These states are primarily characterized by high Arousal (\$y\$-axis). "Overwhelm" is of particular interest computationally; it represents a saturation point where cognitive processing acts as a bottleneck. In the L.O.V.E. system, this is modeled as a high-velocity rotation that exceeds the user's "Elasticity" metric, leading to a state of "flooding" where the system (or the user) cannot process new inputs effectively.⁷ "Vulnerability" is redefined not as weakness, but as the willingness to expose the \$z\$-axis (Connection) despite the risk, making it the gateway to all other connective states.⁹

2.2.2 Places We Go When We Compare

This category includes Comparison, Admiration, Reverence, Envy, Jealousy, Resentment, Schadenfreude, and Freudenfreude.

Comparison creates a relational dependency where the self-vector is defined relative to an external object. "Resentment" is identified as part of this cluster rather than simple anger, often stemming from envy and unmet needs.⁶ In the quaternion space, these emotions typically involve negative Valence (\$x\$) and a blocked or distorted Connection (\$z\$)—the user is connected to the idea of the other person, but disconnected from their own sufficiency.

2.2.3 Places We Go When Things Don't Go As Planned

This category includes Boredom, Disappointment, Expectations, Regret, Discouragement, Resignation, and Frustration.

These states represent a "delta" or error function between an expected trajectory (q_{target}) and the actual trajectory (q_{current}). "Resignation" implies a cessation of rotational momentum—a velocity of zero despite a negative Valence state. "Frustration" implies high Arousal (energy) without movement, a vector spinning in place.¹⁰

2.2.4 Places We Go When It's Beyond Us

This category includes Awe, Wonder, Confusion, Curiosity, Interest, and Surprise.

These are expansive states. "Awe" and "Wonder" facilitate a transcendence of the self, often mapped as high Connection (z) and variable Valence. "Confusion" is a critical state for the Listener module; it represents high entropy or low vector confidence. When the system detects confusion, it signals a need for clarification or slower processing.⁷

2.2.5 Places We Go When Things Aren't What They Seem

This category includes Amusement, Bittersweetness, Nostalgia, Cognitive Dissonance, Paradox, Irony, and Sarcasm.

"Bittersweetness" and "Nostalgia" represent complex quaternions where positive and negative Valence coexist. This challenges simple scalar representations and justifies the use of high-dimensional vector embeddings in pgvector to capture the nuance before projecting to the 3D visualization space. Cognitive dissonance is a state of high internal friction, represented as a vibration in the visualization.¹²

2.2.6 Places We Go When We're Hurting

This category includes Anguish, Hopelessness, Despair, Sadness, and Grief.

These are generally low Arousal, low Valence states. "Anguish" is described as an almost physical pain, suggesting a haptic feedback pattern of heavy, slow throbbing in the Experience module. "Hopelessness" is the absence of a future trajectory, a quaternion with no forward vector.⁸

2.2.7 Places We Go With Others

This category includes Compassion, Pity, Empathy, Sympathy, Boundaries, and Comparative Suffering.

This cluster defines the positive z -axis (Connection). Brown distinguishes clearly between Empathy (feeling with) and Sympathy (feeling for), and between Compassion and Pity. The Versor must distinguish these subtle angular differences; Pity might have a similar Valence to Compassion but a negative or orthogonal Connection vector.⁶

2.2.8 Places We Go When We Fall Short

This category includes Shame, Self-Compassion, Perfectionism, Guilt, Humiliation, and Embarrassment.

"Shame" is the intensely painful feeling of being flawed and unworthy of connection (negative z -axis). "Self-Compassion" is the rotational corrective force that restores the z -axis alignment. "Perfectionism" is a defensive shield, a rigid orientation that resists any modification or rotation.⁷

2.2.9 Places We Go When We Search for Connection

This category includes Belonging, Fitting In, Connection, Disconnection, Insecurity, Invisibility, and Loneliness.

"Belonging" vs. "Fitting In" is a critical distinction. Fitting In requires changing one's orientation to match the environment (external alignment), whereas Belonging is remaining true to one's internal orientation (internal alignment).¹¹ This implies that "Fitting In" has a high dependency on external variables, while "Belonging" is a stable internal state.

2.2.10 Places We Go When the Heart Is Open

This category includes Love, Lovelessness, Heartbreak, Trust, Self-Trust, Betrayal, Defensiveness, Flooding, and Hurt.

"Flooding" is a rapid, overwhelming input that shuts down processing—computationally modeled as a system interrupt or a "freeze" in the state trajectory where input processing (Listener) is temporarily throttled to prevent system crash.¹⁰

2.2.11 Places We Go When Life Is Good

This category includes Joy, Happiness, Calm, Contentment, Gratitude, Foreboding Joy, Relief, and Tranquility.

Brown notes that "Foreboding Joy" is the fear that joy will be taken away—a state of high Valence tainted by high Anxiety (Arousal). This creates a trembling or unstable rotational axis in the visualization. "Gratitude" is noted as the antidote to foreboding joy, stabilizing the rotation.¹³

2.2.12 Places We Go When We Feel Wronged

This category includes Anger, Contempt, Disgust, Dehumanization, Hate, and Self-Righteousness.

"Anger" is an action emotion (high Arousal). "Self-Righteousness" involves rigidity—a resistance to rotation or new data. "Dehumanization" is the complete severing of the Connection axis.¹⁰

2.2.13 Places We Go to Self-Assess

This category includes Pride, Hubris, and Humility.

Hubris is inflated, unconnected pride. Humility is "grounded confidence," suggesting a stable center of gravity in the visualization.¹¹

2.3 Semantic Vector Mapping Table

The following table outlines the preliminary mapping of key emotional clusters to the VAC model. These values serve as the initialization constants for the Observer's vector database.

Category	Representative Emotion	Valence (x)	Arousal (y)	Connection (z)	Vector Reasoning
Uncertainty	Overwhelm	-0.6	+0.9	-0.3	High energy (arousal) but negative tone; connection is frayed due to stress.
Connection	Loneliness	-0.7	-0.2	-0.9	Deeply negative connection; low arousal (depressive state).
Connection	Belonging	+0.8	+0.4	+1.0	High connection is the defining feature; positive valence.
Hurting	Grief	-0.9	-0.4	+0.5	Extremely negative valence; however, grief often involves deep love/connection.
Wronged	Anger	-0.5	+0.8	-0.2	High arousal/action orientation; negative valence; slight disconnection.
Open Heart	Trust	+0.6	+0.1	+0.9	Calm arousal, positive valence, very high connection.
Self-Assess	Hubris	+0.7	+0.6	-0.8	Positive valence (pride), high

					energy, but deep disconnection from reality/others.
--	--	--	--	--	---

3. Mathematical Engine: The Versor

3.1 Why Quaternions?

The representation of 3D orientation is mathematically non-trivial. The most common method, Euler angles (pitch, yaw, roll), suffers from a critical flaw known as "Gimbal Lock." This occurs when two axes align, resulting in the loss of a degree of freedom. In the context of emotional mapping, Gimbal Lock is a potent metaphor for emotional "stuckness"—the state of trauma or flooding where a person loses the ability to pivot or gain perspective.¹⁵ Quaternions avoid this singularity entirely. A quaternion is a hyper-complex number system that allows for the smooth interpolation of rotation across the surface of a 4-dimensional hypersphere. This property makes them ideal for calculating the "shortest path" between two emotional states, which is rarely a straight line in Cartesian space but an arc in conceptual space.

3.2 Quaternion Fundamentals

A quaternion q is defined as:

$$q = w + xi + yj + zk$$

where $w, x, y, z \in \mathbb{R}$ and i, j, k are imaginary units satisfying $i^2 = j^2 = k^2 = ijk = -1$.¹⁷

In the L.O.V.E. system:

- The **Vector Part** ($xi + yj + zk$) encodes the axis of rotation, derived from the normalized VAC inputs (Valence, Arousal, Connection).
- The **Scalar Part** (w) encodes the magnitude of the rotation (the angle θ), representing the intensity of the emotion relative to a neutral baseline.

3.3 State Construction

We model the user's emotional state as a **Unit Quaternion**. The process of converting the normalized scalar inputs from the Listener (v_x, v_y, v_z) into a quaternion $q_{current}$ is as follows:

1. **Input Vector:** $\vec{v} = [v_x, v_y, v_z]$. These are the raw outputs from the semantic analysis.

2. **Magnitude Calculation:** $\|\vec{v}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$. This represents the total intensity of the emotional state.
3. **Axis Normalization:** If $\|\vec{v}\| > 0$, the rotation axis $\hat{u}$ is $\vec{v} / \|\vec{v}\|$.
4. **Angle Determination:** The angle of rotation θ from the neutral identity state is derived from the magnitude. We map the intensity range $[-1, 1]$ to an angular range $[0, \pi]$ (0 to 180 degrees).

$$\theta = \pi \times \|\vec{v}\|$$

5. Quaternion Instantiation:

$$q_{\text{current}} = \cos(\theta/2) + \sin(\theta/2)(u_x i + u_y j + u_z k)$$

This mapping ensures that a neutral state results in the identity quaternion ($1 + 0i + 0j + 0k$), while an intense emotion results in a quaternion representing a significant rotation away from neutral.¹⁶

3.4 Calculating Emotional Work

The concept of "Emotional Work" is quantified as the rotational effort required to transition from a previous state (q_{start}) to a new state (q_{target}).

Transition Quaternion:

To find the rotation that moves q_{start} to q_{target} , we calculate the product of the target and the inverse (conjugate) of the start:

$$q_{\text{transition}} = q_{\text{target}} \times q_{\text{start}}^{-1}$$

Since we use unit quaternions, $q_{\text{start}}^{-1} = q_{\text{start}}^*$ (the conjugate).¹⁷

Angular Distance (ϕ):

The magnitude of the transition—the "size" of the emotional shift—is given by the angle ϕ :

$$\phi = 2 \arccos(|w_{\text{transition}}|)$$

where $w_{\text{transition}}$ is the scalar part of $q_{\text{transition}}$.¹⁹

- Small ϕ : The user is stable or ruminating in a similar state.
- Large ϕ (e.g., $> 90^\circ$): The user is undergoing a radical shift, such as moving from "Despair" to "Hope."

3.5 Spherical Linear Interpolation (SLERP)

To visualize the transition in the Experience module, linear interpolation (LERP) is insufficient as it cuts through the sphere rather than moving along its surface. We use SLERP (Spherical

Linear Interpolation) to calculate the "path" of the soul.¹⁸

$$\text{SLERP}(q_1, q_2, t) = \frac{\sin((1-t)\Omega)}{\sin(\Omega)}q_1 + \frac{\sin(t\Omega)}{\sin(\Omega)}q_2$$

where $\cos(\Omega) = q_1 \cdot q_2$ (the dot product) and t is the interpolation factor ranging from 0 to 1.

SLERP ensures constant angular velocity. This is crucial for the haptic feedback system; a constant velocity allows for consistent vibration patterns that map to the "friction" of the turn.

3.6 Insight Generation: Axis Analysis

The Versor analyzes the axis of rotation of $q_{\text{transition}}$ to generate natural language insights.

- **Pure Valence Rotation (x -axis):** "You are feeling better/worse, but your energy is constant."
- **Pure Arousal Rotation (y -axis):** "You are shifting energy levels. Try to ground yourself."
- **Pure Connection Rotation (z -axis):** "You are moving toward or away from others."

If the rotation is complex (involving all three axes), the system identifies the dominant component. For example, a shift from Anger to Sadness might involve a reduction in Arousal (y) and a shift in Valence (x), but the "Connection" might remain low.

4. System Architecture: The L.O.V.E. Stack

The architecture is designed as a reactive loop. The user provides input to the Listener, which normalizes it for the Observer. The Observer provides context to the Versor, which computes the state change. The Versor's output drives the Experience, which provides feedback to the user, closing the loop.

4.1 L - The LISTENER (Ingestion & Normalization)

The Listener is the sensory threshold of the system. Its primary responsibility is to convert unstructured human signals (voice, text) into structured, normalized vectors.

4.1.1 Audio Processing Strategy: Hybrid Transcriptions

The requirement for "Voice Notes" necessitates a robust Speech-to-Text (STT) pipeline. A critical trade-off exists between latency and accuracy.²¹ Cloud-based APIs (like OpenAI Whisper) offer high accuracy but introduce latency and privacy concerns. On-device models offer instant feedback but lower accuracy.

Architectural Decision: A Hybrid Approach.

- **Real-Time (Edge):** For immediate visual feedback and short commands, the system utilizes **whisper.rn**, a React Native binding for **whisper.cpp**. This runs a quantized version of the Whisper model (e.g., **base.en** or **tiny.en**) directly on the user's device. This ensures zero-latency feedback for the visualization and keeps short, intimate utterances

private.³

- **Asynchronous (Cloud):** For detailed journal entries and deep analysis, the audio is uploaded to a secure backend service running **FastAPI**. This service utilizes faster-whisper on GPU instances to transcribe long-form audio with maximum accuracy (large-v3 model).²⁴

4.1.2 Semantic Analysis: LLM & LangChain

Once text is transcribed, it must be parsed into the VAC scalars.

- **Orchestration: LangChain** is used to manage the interaction with the Large Language Model (LLM).
- **Model:** A lightweight, fast-inference model (e.g., Llama 3-8B via Groq or GPT-4o-mini) is sufficient for this classification task.
- **Schema Enforcement:** To ensure the Versor receives valid mathematical inputs, we employ **Pydantic** parsers within LangChain. This forces the LLM to output a strict JSON structure, preventing "hallucinations" in formatting.²⁵

Prompt Strategy:

The prompt must include the "Atlas" definitions as few-shot examples.

"Analyze the following text. Based on Brené Brown's Atlas of the Heart, map the emotional state to three scalars (-1.0 to 1.0): Valence (Pleasure), Arousal (Energy), Connection (Relational Alignment). Output purely JSON."

4.1.3 PII Sanitization

Before any data is passed to the Observer or stored permanently, the Listener must strip Personally Identifiable Information (PII). This is handled via a named-entity recognition (NER) pass (using Spacy or a specialized LLM prompt) to replace names, locations, and dates with generic tokens (e.g., , ,).

4.2 O - The OBSERVER (Context & State Management)

The Observer is the system's memory. It persists the user's trajectory and holds the definitions of the emotional universe.

4.2.1 Database Technology: PostgreSQL + pgvector

The core data store is **PostgreSQL**, enhanced with the **pgvector** extension. This allows for the storage of both relational data (users, sessions) and high-dimensional vector embeddings in the same ACID-compliant database.²⁷

4.2.2 Schema Design

The schema requires two primary tables: one for the static definitions and one for the dynamic user history.

Table: atlas_definitions

This table stores the 87 emotions.

- id: Primary Key.

- **emotion_name:** String (e.g., "Schadenfreude").
- **category:** String (e.g., "Places We Go When We Compare").
- **definition:** Text.
- **embedding:** vector(1536) - A high-dimensional semantic vector (generated by OpenAI text-embedding-3-small) used for semantic search.
- **vac_vector:** vector(3) - The normalized [Valence, Arousal, Connection] vector.
- **q_constant:** vector(4) - The pre-calculated quaternion representation [w, x, y, z].

Table: user_trajectory

This table logs the user's journey.

- **id:** UUID.
- **user_id:** UUID (Foreign Key).
- **timestamp:** TIMESTAMPTZ.
- **input_transcription:** Text (Sanitized).
- **vac_values:** vector(3) - The calculated input.
- **quaternion_state:** vector(4) - The resulting state.
- **elasticity_metric:** Float - A derived metric indicating the speed of change.

4.2.3 Indexing Strategy

Performance is critical for the "Insight Generation" feature, which looks up similar past states.

- **Atlas Definitions:** With only 87 rows, no vector index is needed; a sequential scan is instant.
- **User Trajectory:** As this table grows, finding "nearest neighbors" (past moments like the current one) requires indexing.
 - **Recommendation:** Use **HNSW (Hierarchical Navigable Small World)** indexing for the quaternion_state column. HNSW offers superior query performance (low latency) compared to IVFFlat, especially for high-recall requirements, at the cost of slightly slower build times.⁴
 - **Configuration:** For datasets under 1 million rows, standard parameters (m=16, ef_construction=64) are sufficient.²⁹

4.3 V - The VERSOR (The Mathematical Engine)

The Versor is a stateless microservice dedicated to calculation. It decouples the math from the state (Observer) and the UI (Experience).

4.3.1 Tech Stack

- **Language:** Python (for its rich scientific computing ecosystem).
- **Libraries:** numpy for vector operations, scipy.spatial.transform for robust rotation and SLERP implementations.
- **Interface:** GRPC or REST (FastAPI) for communication with the Listener and Experience modules.

4.3.2 Logic Flow

1. **Input:** Receives current_vac from Listener and previous_quaternion from Observer.

2. Processing:

- Converts current_vac to target_quaternion.
- Computes transition_quaternion ($q_{\text{trans}} = q_{\text{target}} \cdot q_{\text{prev}}^*$).
- Computes angular distance θ .
- Generates a SLERP path (a list of 60-120 intermediate quaternions) for the UI to render.

3. **Elasticity Calculation:** Calculates $E = \theta / \Delta t$. If E exceeds a threshold, it flags the transition as "High Velocity," triggering specific guidance in the Experience module.

4.4 E - The EXPERIENCE (Visualization & Interface)

The Experience module is the user-facing application. It must render the abstract mathematics as a visceral, intuitive reality.

4.4.1 Framework: React Native + React Three Fiber

The application is built using **React Native** to ensure a native mobile experience. For 3D rendering, **React Three Fiber (R3F)** is the industry standard for declarative WebGL/OpenGL within React. R3F allows the "Soul Sphere" to be managed as a standard React component, with its rotation driven by props derived from the Versor's output.⁵

Note on Compatibility: The React Native ecosystem is currently transitioning to a "New Architecture" (Fabric/TurboModules). Historical compatibility issues exist between R3F, Expo GL, and this New Architecture.³²

- **Risk Mitigation:** The project should explicitly target **Expo SDK 52+** and utilize the `@react-three/fiber/native` bindings. If instability is detected with the New Architecture enabled, the 3D viewing components should be isolated or the app should temporarily opt-out of the New Architecture via `gradle.properties` (`newArchEnabled=false`) until full stability is confirmed.³³

4.4.2 Visualization Logic: The Soul Sphere

The user is represented by a dynamic sphere.

- **Geometry:** A `SphereGeometry` with high segment count for smoothness.
- **Material:** A `MeshStandardMaterial` or custom `ShaderMaterial`.
 - **Color:** Mapped to Valence (Red = Negative, Blue/Green = Positive).
 - **Roughness/Displacement:** Mapped to Arousal. High arousal adds noise vertex displacement, making the sphere "spiky" or vibrating.
 - **Opacity/Glow:** Mapped to Connection. High connection makes the sphere translucent and radiant; disconnection makes it opaque and dull.

4.4.3 Haptics and Bio-Feedback

To simulate the "friction" of emotional work, the app utilizes haptic feedback.

- **Library:** `react-native-haptics` (or `expo-haptics`) provides access to the Taptic Engine on iOS and Vibrator on Android.³⁴

- **Pattern Generation:**
 - **High Resistance ($\theta > 90^\circ$):** As the sphere rotates through the midpoint of a large transition, the phone generates a "Heavy" impact (`Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Heavy)`), simulating the effort of turning a heavy wheel.
 - **Stability ($\theta \approx 0^\circ$):** A subtle, heartbeat-like pulse indicates the user is maintaining a state.
 - **Flooding (High Elasticity):** A rapid, chaotic vibration pattern warns the user of emotional overwhelm.

4.4.4 Optimization for Mobile Performance

Rendering 3D scenes on mobile devices is resource-intensive and can drain battery.

- **Frameloop on Demand:** The `<Canvas>` component should be set to `frameloop="demand"`. This ensures the renderer only ticks when a prop changes (i.e., during a rotation animation) rather than running a continuous 60fps loop when the user is idle.⁵
- **Geometry Instancing:** The sphere geometry should be created once and memoized using `useMemo` to prevent garbage collection churn during re-renders.³⁶
- **Texture Management:** Any textures used for the environment or sphere surface must be compressed (e.g., KTX2) to reduce GPU memory bandwidth usage.

5. Detailed Development Roadmap

Phase 1: The Versor Core (MVP - Weeks 1-4)

Goal: Prove the mathematics and rotational logic.

- **Task 1.1:** Set up a Python development environment with `numpy`, `numpy-quaternion`, and `scipy`.
- **Task 1.2:** Implement the Quaternion class wrapping the VAC-to-Quaternion logic.
- **Task 1.3:** Create a test suite of "Canonical Transitions":
 - *Test A:* Anger (-0.5, 0.8, -0.2) to Calm (0.8, -0.6, 0.5). Verify θ is large.
 - *Test B:* Shame (-0.9, -0.1, -1.0) to Guilt (-0.8, -0.1, -0.8). Verify θ is small (Nuance check).
- **Task 1.4:** Implement the SLERP function and generate path arrays.

Phase 2: The Listener & Observer (Integration - Weeks 5-8)

Goal: Connect unstructured reality to structured data.

- **Task 2.1:** Provision a PostgreSQL instance with `pgvector` enabled (e.g., Supabase or AWS RDS).
- **Task 2.2:** Design the database schema (`atlas_definitions`, `user_trajectory`) and ingest the 87 emotional definitions.

- **Task 2.3:** Build the FastAPI Listener service.
- **Task 2.4:** Integrate LangChain with an LLM provider (OpenAI or Groq) to implement the "Text-to-VAC" extraction chain with Pydantic validation.
- **Task 2.5:** Integrate whisper.rn into a basic React Native shell to test on-device transcription accuracy.

Phase 3: The Experience (Visual Proof - Weeks 9-12)

Goal: See the math in motion.

- **Task 3.1:** Initialize the full React Native project (Expo workflow recommended for ease of library integration).
- **Task 3.2:** Implement the R3F <Canvas> and the SoulSphere component.
- **Task 3.3:** Wire the Versor API to the UI. When the API returns a SLERP path, the UI must animate the sphere through the sequence of quaternions.
- **Task 3.4:** Implement the "color/roughness/opacity" shaders to react to VAC values dynamically.

Phase 4: Optimization & Polish (Weeks 13-16)

Goal: Refine the feeling and performance.

- **Task 4.1: Haptic Tuning:** rigorous testing of vibration patterns to ensure they feel "supportive" rather than "annoying."
- **Task 4.2: Performance Profiling:** Use Flipper and React DevTools to identify render bottlenecks. Ensure the animation maintains 60fps on mid-range devices.
- **Task 4.3: Prompt Engineering:** Refine the LLM system prompts based on beta testing. If "Frustration" is constantly being mislabeled as "Anger," adjust the few-shot examples in the prompt.
- **Task 4.4: Security Audit:** Verify PII stripping and secure API key management.

6. Challenges and Risk Management

6.1 The "Connection" Mapping Ambiguity

Risk: While Valence and Arousal are well-documented in psychological literature (Russell's Circumplex), "Connection" is a novel dimension introduced by Project L.O.V.E. There is no pre-existing dataset training LLMs to recognize "Connection" scores explicitly.

Mitigation: We must create a "Ground Truth" lookup table. The team will manually annotate the 87 Atlas emotions with Connection scores (e.g., Loneliness = -1.0, Belonging = 1.0). This table will be injected into the LLM's context window (RAG or Few-Shot) to align its scoring with our proprietary model.³⁷

6.2 Quaternion User Comprehension

Risk: Users will not understand what a "Quaternion" is. Displaying raw data (\$w=0.7\$) will be

alienating.

Mitigation: The interface must never show the math. It must translate the math. The Versor includes a "Translation Layer" that converts axis data into sentences. If the user rotates purely on the z -axis, the UI says: "You are reconnecting."

6.3 Real-Time Latency

Risk: The round trip (Voice -> Edge/Cloud -> LLM -> DB -> Math -> UI) could take 3-5 seconds, breaking the sense of immersion.

Mitigation:

1. **Optimistic Updates:** The UI should begin a generic "processing" animation (e.g., a gentle pulse) immediately upon voice detection.
2. **Streaming Architecture:** If possible, stream the text from `whisper.rn` to the LLM token-by-token, allowing the Versor to calculate a preliminary vector before the sentence is finished.²¹

7. Conclusion

Project L.O.V.E. is an ambitious synthesis of psychology and engineering. By leveraging the non-linear properties of quaternions, it offers a mathematical model that honors the complexity of the human spirit. It moves beyond the flat, static maps of traditional analytics and provides a dynamic, three-dimensional compass for navigating the emotional landscape. With a robust architecture built on the L.O.V.E. Stack—combining the edge-processing of the Listener, the vector memory of the Observer, the algebraic logic of the Versor, and the immersive visualization of the Experience—the system is poised to define a new standard in affective computing.

Works cited

1. Emotions List at a Glance - Sources of Insight, accessed December 2, 2025, <https://sourcesofinsight.com/emotions-list/>
2. Brene Brown Emotions, accessed December 2, 2025, <https://www.depts.ttu.edu/rise/Posters/BreneBrownEmotions.pdf>
3. mybigday/whisper.rn: React Native binding of whisper.cpp. - GitHub, accessed December 2, 2025, <https://github.com/mybigday/whisper.rn>
4. Supercharging vector search performance and relevance with pgvector 0.8.0 on Amazon Aurora PostgreSQL | AWS Database Blog, accessed December 2, 2025, <https://aws.amazon.com/blogs/database/supercharging-vector-search-performance-and-relevance-with-pgvector-0-8-0-on-amazon-aurora-postgresql/>
5. React-Three-Fiber: Enhancing Scene Quality with Drei + Performance Tips - Medium, accessed December 2, 2025, <https://medium.com/@ertugrulyaman99/react-three-fiber-enhancing-scene-quality-with-drei-performance-tips-976ba3fba67a>
6. Book Summary – Atlas of the Heart: Mapping Meaningful Connection and the

- Language of Human Experience - Readinggraphics, accessed December 2, 2025,
<https://readinggraphics.com/book-summary-atlas-of-the-heart/>
7. Atlas of the Heart by Brené Brown – Here are my six lessons and takeaways, accessed December 2, 2025,
<https://www.15minutebusinessbooks.com/blog/2022/01/24/atlas-of-the-heart-by-brene-brown-here-are-my-six-lessons-and-takeaways/>
 8. 87 Human Emotions and Experiences - 1page | PDF - Scribd, accessed December 2, 2025,
<https://www.scribd.com/document/656659375/87-Human-Emotions-and-Experiences-1Page>
 9. Atlas of the Heart Summary - Four Minute Books, accessed December 2, 2025,
<https://fourminutebooks.com/atlas-of-the-heart-summary/>
 10. Atlas of the Heart by Brené Brown - Ryan Delaney, accessed December 2, 2025,
<https://www.ryandelaney.co/book-notes/atlas-of-the-heart-summary>
 11. [PDF] Atlas of the Heart Summary - Brené Brown - Shortform, accessed December 2, 2025,
<https://www.shortform.com/pdf/atlas-of-the-heart-pdf-brene-brown>
 12. How to optimize performance when using pgvector - Azure Cosmos DB for PostgreSQL, accessed December 2, 2025,
<https://learn.microsoft.com/en-us/azure/cosmos-db/postgresql/howto-optimize-performance-pgvector>
 13. Atlas of the Heart - Brené Brown, accessed December 2, 2025,
<https://brenebrown.com/book/atlas-of-the-heart/>
 14. Book notes: Atlas of the Heart by Brené Brown - Marlo Yonocruz, accessed December 2, 2025,
<https://marloyonocruz.com/2022/10/28/book-notes-atlas-of-the-heart-by-brene-brown/>
 15. Orientation & Quaternions, accessed December 2, 2025,
https://cseweb.ucsd.edu/classes/wi19/cse169-a/slides/CSE169_03.pdf
 16. Quaternion-Based Signal Analysis for Motor Imagery Classification from Electroencephalographic Signals - MDPI, accessed December 2, 2025,
<https://www.mdpi.com/1424-8220/16/3/336>
 17. Rotations, Orientation, and Quaternions - MATLAB & Simulink - MathWorks, accessed December 2, 2025,
<https://www.mathworks.com/help/fusion/ug/rotations-orientation-and-quaternions.html>
 18. Quaternions and Rotations* - Stanford Computer Graphics Laboratory, accessed December 2, 2025,
<https://graphics.stanford.edu/courses/cs348a-17-winter/Papers/quaternion.pdf>
 19. dist - Angular distance in radians - MATLAB - MathWorks, accessed December 2, 2025,
<https://www.mathworks.com/help/driving/ref/quaternion.dist.html>
 20. accessed December 2, 2025,
[https://www.mathworks.com/help/driving/ref/quaternion.dist.html#:~:text=The%20angular%20distance%20between%20two%20quaternions%20can%20be%20expressed%20as.\(%20real%20\(%20z%20\)%20\)%20](https://www.mathworks.com/help/driving/ref/quaternion.dist.html#:~:text=The%20angular%20distance%20between%20two%20quaternions%20can%20be%20expressed%20as.(%20real%20(%20z%20)%20)%20)

21. Why Enterprises Are Moving to Streaming — and Why Whisper Can't Keep Up - Deepgram, accessed December 2, 2025, <https://deepgram.com/learn/why-enterprises-are-moving-to-streaming-and-why-whisper-can-t-keep-up>
22. Building an AI-Powered Note-Taking App in React Native - Part 4: Automatic Speech Recognition | Software Mansion, accessed December 2, 2025, <https://blog.swmansion.com/building-an-ai-powered-note-taking-app-in-react-native-part-4-automatic-speech-recognition-a3b50e7d2245>
23. How to Use OpenAI's Whisper in React Native - YouTube, accessed December 2, 2025, <https://www.youtube.com/watch?v=FLTVh6ZBTa4>
24. Host the Whisper Model with Streaming Mode on Amazon EKS and Ray Serve | Containers, accessed December 2, 2025, <https://aws.amazon.com/blogs/containers/host-the-whisper-model-with-streaming-mode-on-amazon-eks-and-ray-serve/>
25. Structured output - Docs by LangChain, accessed December 2, 2025, <https://docs.langchain.com/oss/python/langchain/structured-output>
26. Mastering Structured Output in LangChain: Pydantic, TypedDict, and JSON Schema | by A S M Morshedul Hoque (Utsho) | Medium, accessed December 2, 2025, <https://medium.com/@asmmorshedulhoque/mastering-structured-output-in-langchain-pydantic-typeddict-and-json-schema-573d67d5daa4>
27. pgvector/pgvector: Open-source vector similarity search for Postgres - GitHub, accessed December 2, 2025, <https://github.com/pgvector/pgvector>
28. Faster similarity search performance with pgvector indexes | Google Cloud Blog, accessed December 2, 2025, <https://cloud.google.com/blog/products/databases/faster-similarity-search-performance-with-pgvector-indexes>
29. Performance Tips Using Postgres and pgvector | Crunchy Data Blog, accessed December 2, 2025, <https://www.crunchydata.com/blog/pgvector-performance-for-developers>
30. Understanding PostgreSQL pgvector Indexing with IVFFlat | by Mauricio Maia | Medium, accessed December 2, 2025, <https://medium.com/@mauricio/optimizing-ivfflat-indexing-with-pgvector-in-postgresql-755d142e54f5>
31. The current state of using React Three Fiber in React Native + Expo | by Trifon Statkov, accessed December 2, 2025, <https://trifonstatkov.medium.com/the-current-state-of-using-react-three-fiber-in-react-native-expo-c65918593eaf>
32. My experience with Expo 52 and R3F and Drei (RANT) : r/reactnative - Reddit, accessed December 2, 2025, https://www.reddit.com/r/reactnative/comments/1iaaon3/my_experience_with_expo_52_and_r3f_and_drei_rant/
33. About the New Architecture - React Native, accessed December 2, 2025, <https://reactnative.dev/architecture/landing-page>
34. Haptics - Expo Documentation, accessed December 2, 2025,

<https://docs.expo.dev/versions/latest/sdk/haptics/>

35. React Native Haptics: A Fast, High-Performance Library for iOS Haptics and Android Vibration Effects | by M.H.Pousti | Medium, accessed December 2, 2025, <https://medium.com/@hpousty/react-native-haptics-a-fast-high-performance-library-for-ios-haptics-and-android-vibration-ec107f97a7ee>
36. Performance pitfalls - React Three Fiber, accessed December 2, 2025, <https://r3f.docs.pmnd.rs/advanced/pitfalls>
37. A Proxy-Based Method for Mapping Discrete Emotions onto VAD model - arXiv, accessed December 2, 2025, <https://arxiv.org/pdf/2511.12521>